

Founding Engineer — Home Assignment

Time: 3–4 hours **Format:** Three documents (structure provided below)

Background

You're working on a multi-tenant data platform. Tenants define their own derived datasets on top of shared base schemas they cannot modify. Every dataset field has two definitions:

- **Syntactic** — name and type
- **Semantic** — what it means (units, uniqueness guarantees, null interpretation, time semantics)

Derived datasets can depend on other derived datasets. The system tracks lineage between all of them.

The Incident

The platform has been live for a year. There are hundreds of derived datasets, both batch and streaming, with historical data already stored.

You discover a field in a shared base dataset has an incorrect semantic definition:

Field: `event_timestamp` **Was defined as:** Free-text string (e.g. "March 5, 2024 - 3pm") **Actually is:** ISO 8601 date (YYYY-MM-DD, UTC, no time component)

This field is used in global Daily Active Users metrics, cross-region aggregations, billing deduplication, ML feature tables, and identity joins. Correcting it changes the meaning of historical data across potentially millions of records.

Your Task

Design how the platform handles this correction and its propagation. This is not a schema migration — it's about safely updating meaning across a live dependency graph of materialized data.

Deliver Three Documents

Document 1 — Data Models

Define the database tables that support this system. For each table, list the columns and explain in 1–2 sentences what it represents and why it exists.

Cover at minimum:

- *How semantic definitions are stored and versioned*
- *How derived datasets bind to a specific version of a field's semantic definition*
- *How lineage (dataset-to-dataset dependencies) is represented*
- *How you track the status of a propagation event (what changed, what's been recomputed, what's pending)*

You don't need SQL DDL — a clean table-and-columns description is fine.

Document 2 — AWS Architecture

Describe the AWS services you'd use and how they connect. A diagram description or a written walkthrough is both fine — focus on clarity.

Cover:

- *Where semantic definitions and lineage metadata are stored*
- *What gets triggered when a semantic definition changes*
- *How recomputation jobs are orchestrated (batch and streaming separately)*
- *How tenant isolation is maintained*
- *How rollback is supported*

Call out any meaningful tradeoffs in your service choices (e.g. why Step Functions vs EventBridge, why S3 versioning vs a separate table).

Document 3 — Update Flow

Walk through what happens, step by step, when an engineer corrects the `event_timestamp` semantic definition.

Structure it as a sequence: what fires, what checks, what computes, in what order. Cover:

- How the system detects which datasets are affected (directly and indirectly)
- What the recomputation order is and why
- What happens to streaming pipelines vs batch materializations
- How you prevent partial inconsistency while propagation is in progress
- What downstream tenants see during and after the update
- How you'd roll back if something goes wrong

End with a short section (half a page max) on the core tradeoffs you made: correctness vs cost, eager vs lazy recomputation, versioned datasets vs in-place updates.

How We'll Evaluate This

We're not looking for a perfect design — we're looking for evidence that you own your decisions.

Specifically:

- **Explain your choices.** If you used an AI tool to help generate a solution, what did you change and why? We want to see your judgment, not the first output.
- **Walk us through your thinking.** What options did you consider for each decision, and what made you pick one over the others?
- **Know your own plan.** Be ready to explain the flows you designed — what triggers what, in what order, and why it's structured that way.
- **Surface the hard parts.** Where did you get stuck? What would you do differently with more time?